1 ВВЕДЕНИЕ

1.1 Цель

Документ описывает требования к программе **CacheProxy** — HTTP-прокси-серверу с кешированием и фильтрацией URL. По этим требованиям команда разрабатывает и проверяет программу.

Команда проекта (группа ЭФБО-04-25): Коптилин Илья Русланович, Кирбитов Артем Евгеньевич, Звягин Тимур Олегович.

1.2 Область действия (Scope)

CacheProxy — консольная программа на C++, которую ставят между компьютерами локальной сети (например, учебного класса) и интернетом. Программа:

- принимает HTTP-запросы пользователей и пересылает их на сайты;
- сохраняет ответы в кеш и при повторном запросе отдаёт их сразу, без обращения к сайту;
- блокирует запрещённые сайты по правилам администратора;
- пропускает HTTPS-сайты, не расшифровывая их.

В проект не входят: расшифровка HTTPS, графический интерфейс, учётные записи пользователей.

1.3 Определения и аббревиатуры

Таблица 1 – Определения и аббревиатуры

Термин	**Значение**
Прокси-сервер	Программа-посредник: получает запрос пользователя и сама обращается к сайту.
Кеш	Хранилище сохранённых ответов сайтов для повторного использования.
HIT / MISS	Ответ найден в кеше / ответа в кеше нет, нужно идти на сайт.
URL	Адрес страницы, например http://site.ru/page.
HTTP / HTTPS	Протокол обмена с сайтами / то же, но с шифрованием.
CONNECT	Способ пропустить HTTPS через прокси: прокси просто передаёт зашифрованные данные.
SRS	Software Requirements Specification — документ с требованиями к программе.
UML	Язык диаграмм для описания программ.

2 ОБЩЕЕ ОПИСАНИЕ

2.1 Описание продукта

Пользователь указывает адрес прокси в настройках браузера. Дальше каждый запрос проходит так:

1) прокси проверяет адрес сайта по правилам блокировки;
2) если сайт запрещён — показывает страницу «Сайт заблокирован»;
3) если ответ уже есть в кеше и он не устарел — сразу отдаёт его;
4) иначе загружает страницу с сайта, сохраняет в кеш и отдаёт пользователю.

2.2 Функции продукта

– пересылка HTTP-запросов;
– кеширование ответов;
– блокировка сайтов по правилам;
– пропуск HTTPS-сайтов;
– статистика и журнал запросов для администратора.

2.3 Характеристики пользователей

– **Пользователь** — работает в браузере, о прокси не задумывается. Ему важно, чтобы сайты открывались быстро и было понятно, почему сайт заблокирован.
– **Администратор** — запускает прокси, настраивает правила блокировки, смотрит статистику. Умеет работать с командной строкой.

3 ДЕТАЛЬНЫЕ ТРЕБОВАНИЯ

3.1 Методология сбора требований

Использованы два метода:

– **интервью с заказчиком** — администратором учебного компьютерного класса (роль заказчика смоделирована). Из интервью выяснили: канал перегружен, потому что 30 студентов скачивают одно и то же; во время занятий нужно блокировать развлекательные сайты; менять правила нужно без перезапуска прокси;

– **User Stories** — требования записаны от лица пользователей в виде «Как <роль>, я хочу <действие>, чтобы <цель>» и разделены по важности методом MoSCoW: Must — обязательно, Should — желательно.

3.2 Бизнес-требования

Таблица 2 – Бизнес-требования

ID	Требование
BR-1	Сократить интернет-трафик: не менее 30 % повторных запросов должны обслуживаться из кеша.
BR-2	Ускорить повторное открытие страниц.
BR-3	Дать администратору контроль над тем, какие сайты доступны.

3.3 Пользовательские требования

Таблица 3 – Пользовательские истории

ID	История	Приоритет
US-1	Как пользователь, я хочу открывать сайты через прокси, чтобы работать в интернете как обычно.	Must
US-2	Как пользователь, я хочу, чтобы повторно открываемые страницы загружались быстрее.	Must
US-3	Как пользователь, я хочу открывать HTTPS-сайты через тот же прокси.	Must
US-4	Как пользователь, я хочу видеть понятное сообщение, если сайт заблокирован.	Should
US-5	Как администратор, я хочу задавать список запрещённых сайтов.	Must
US-6	Как администратор, я хочу менять правила без перезапуска прокси, чтобы не обрывать загрузки.	Should
US-7	Как администратор, я хочу видеть статистику и журнал запросов.	Should

3.4 Функциональные требования

Таблица 4 – Функциональные требования

ID	Требование
FR-1	Прокси принимает подключения на порту из настроек (по умолчанию 8080).
FR-2	Прокси разбирает HTTP-запрос и пересылает его на нужный сайт.
FR-3	Прокси обслуживает несколько пользователей одновременно.
FR-4	Прокси сохраняет в кеш ответы на GET-запросы, если сайт это разрешает (заголовок Cache-Control).
FR-5	Свежий ответ из кеша отдаётся без обращения к сайту.
FR-6	Когда кеш заполнен, удаляются записи, которые дольше всего не использовались.
FR-7	Прокси загружает правила блокировки из файла (домен или маска адреса).
FR-8	На запрещённый сайт прокси отвечает кодом 403 и страницей «Сайт заблокирован».
FR-9	Правила можно перечитать командой `reload` без перезапуска.
FR-10	HTTPS-сайты пропускаются через метод CONNECT без расшифровки.
FR-11	Каждый запрос записывается в журнал: время, IP, адрес, код ответа, взят ли из кеша.
FR-12	Команда `stats` показывает число запросов и долю ответов из кеша.

3.5 Нефункциональные требования

Таблица 5 – Нефункциональные требования

ID	Требование
NFR-1	Производительность: ответ из кеша — не дольше 10 мс.
NFR-2	Нагрузка: не меньше 50 пользователей одновременно.
NFR-3	Надёжность: ошибка в одном запросе не роняет всю программу.
NFR-4	Безопасность: прокси принимает подключения только из локальной сети.
NFR-5	Совместимость: работает в Linux и Windows.
NFR-6	Реализация: язык C++17; разбор HTTP, кеш и фильтр пишутся самостоятельно.

4 UML-ДИАГРАММЫ

4.1 Диаграмма вариантов использования

Показывает, кто работает с системой и что может делать (рисунок 1).

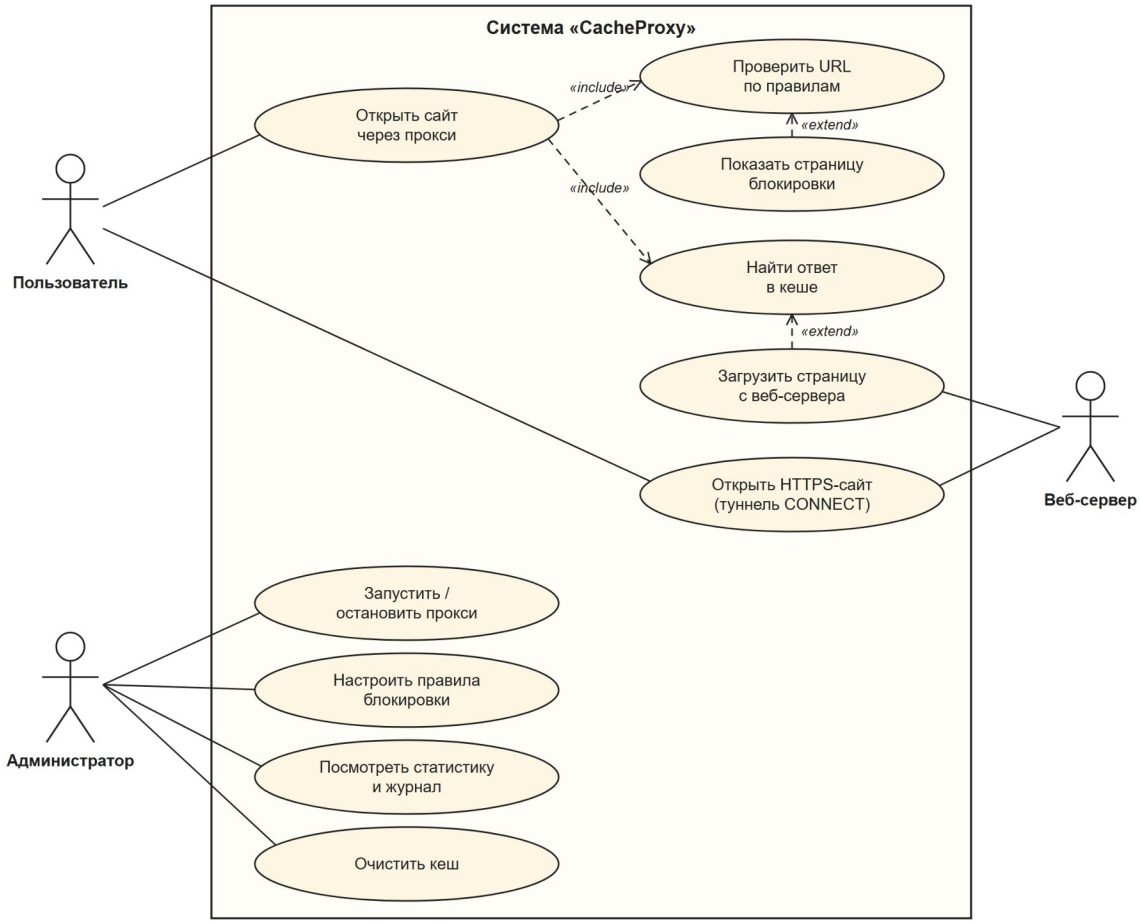


Рисунок 1 – Диаграмма вариантов использования

- **Пользователь** открывает сайты, в том числе HTTPS.
- **Администратор** управляет прокси: запуск, правила, статистика, очистка кеша.
- **Веб-сервер** — внешняя система, с которой прокси загружает страницы.
- «include» — действие выполняется всегда (каждый запрос проверяется фильтром и ищется в кеше); «extend» — только при условии (страница блокировки — если сайт запрещён, загрузка с сервера — если ответа нет в кеше).

4.2 Диаграмма классов

Показывает основные классы программы и связи между ними (рисунок 2).

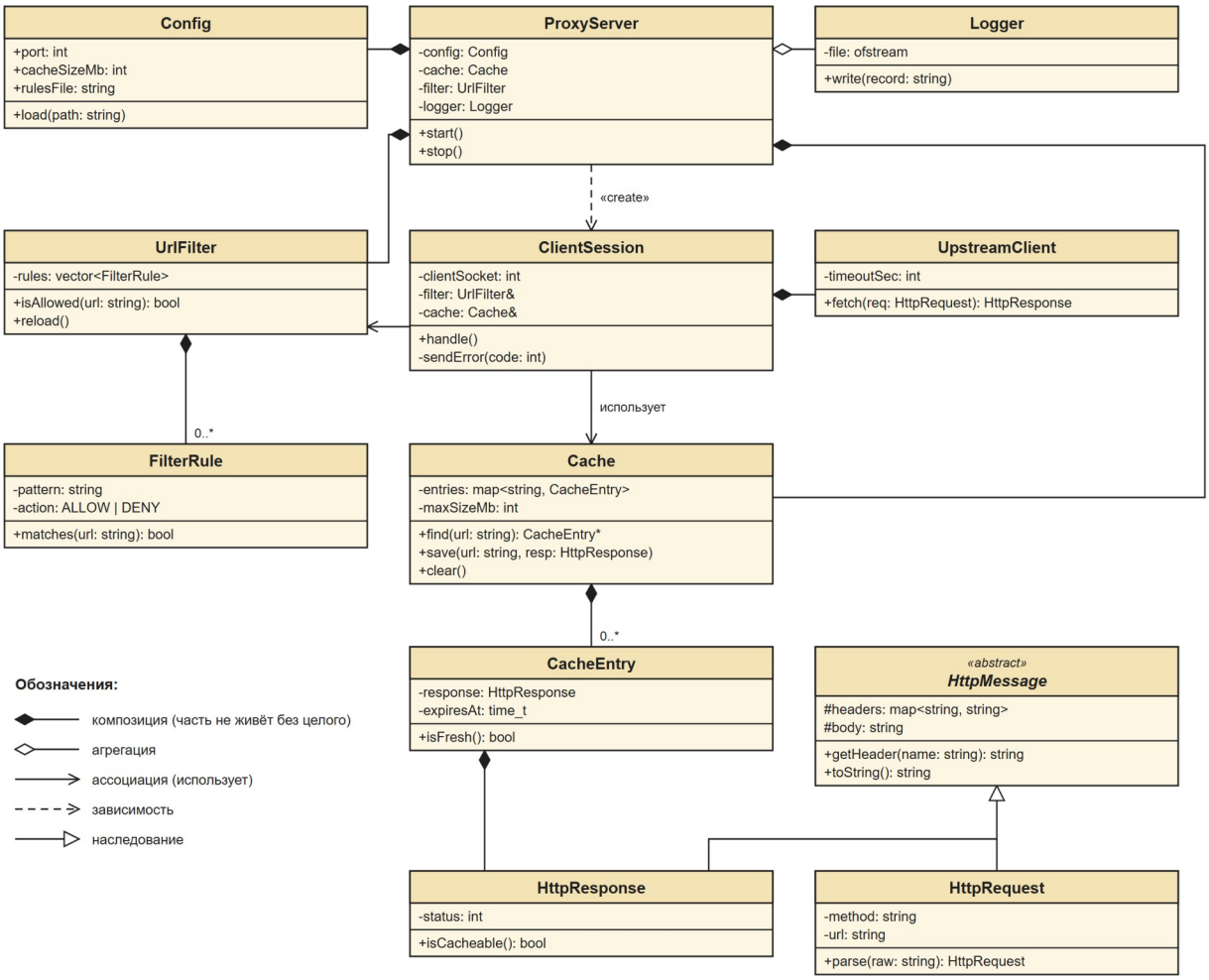


Рисунок 2 – Диаграмма классов

– **ProxyServer** — главный класс: хранит настройки, кеш, фильтр и журнал; на каждое подключение создаёт **ClientSession**.
– **ClientSession** обрабатывает один запрос: проверяет его в **UrlFilter**, ищет в **Cache**, при необходимости загружает страницу через **UpstreamClient**.
– **Cache** хранит записи **CacheEntry**; **UrlFilter** хранит правила **FilterRule**.
– **HttpRequest** и **HttpResponse** наследуются от общего класса **HttpMessage** (заголовки и тело).

4.3 Диаграмма последовательности

Показывает по шагам, как обрабатывается один запрос (рисунок 3).

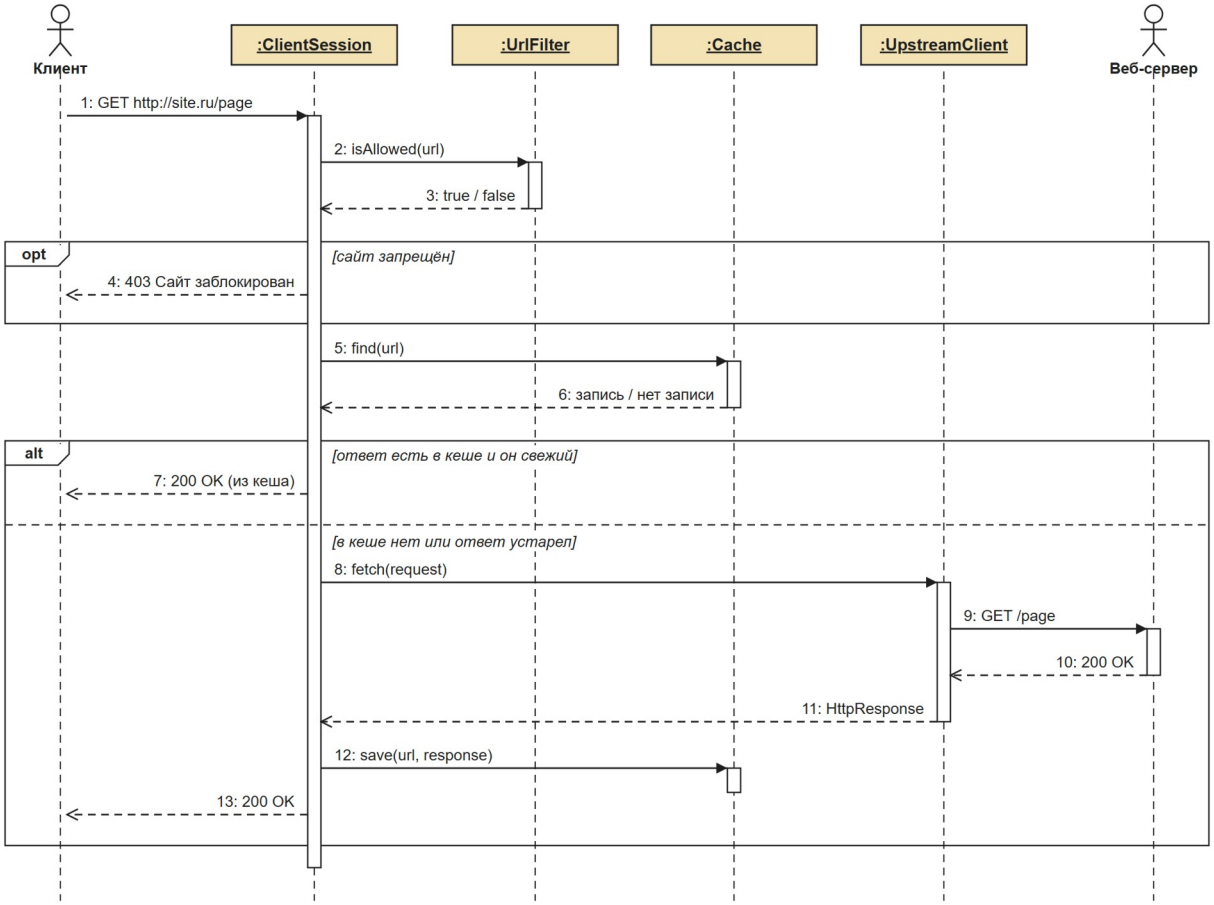


Рисунок 3 – Диаграмма последовательности обработки запроса

1) Клиент отправляет запрос, ClientSession проверяет адрес в UrlFilter (шаги 1–3).
2) Блок **opt**: если сайт запрещён, клиент получает 403 (шаг 4).
3) ClientSession ищет ответ в Cache (шаги 5–6).
4) Блок **alt**: если ответ в кеше свежий — он сразу отдаётся клиенту (шаг 7); иначе страница загружается с веб-сервера, сохраняется в кеш и отдаётся клиенту (шаги 8–13).